ROS 2 Control

A Complete Tutorial

From Basics to a Fully Controlled
Four-Wheel Robot in Gazebo

What this tutorial covers

ROS 2 Control architecture · Hardware interfaces · Controllers
Controller Manager · Custom ROS 2 Service definition
Four-wheel URDF with inertia · Gazebo Classic simulation
Differential drive kinematics · Keyboard teleoperation

ROS 2 Humble · Ubuntu 22.04 · Gazebo Classic 11

Prerequisites: ROS 2 Humble installed, workspace at `~/ros2_ws`, basic terminal familiarity

Contents

1 What Is ROS 2 Control? 2
- 1.1 Problems it solves . . . 2

2 Architecture Overview 2
- 2.1 Key components . . . 3
- 2.2 Controller lifecycle . . . 3

3 Key Concepts 3
- 3.1 Command Interfaces vs State Interfaces . . . 3
- 3.2 Controller types used in this tutorial . . . 3

4 Install Required Packages 4

5 Create the ROS 2 Package 4
- 5.1 CMakeLists.txt . . . 4
- 5.2 package.xml . . . 5

6 Write the URDF 5
- 6.1 Materials . . . 5
- 6.2 Chassis (base_link) . . . 6
- 6.3 Front wheels (fixed — passive) . . . 6
- 6.4 Rear wheels (continuous — driven) . . . 7
- 6.5 Lidar sensor link . . . 8

7 Add the ros2_control Block to the URDF 9

8 Controllers Configuration 10
- 8.1 Differential drive kinematics . . . 10

9 Create a Custom ROS 2 Service 11
- 9.1 Topic vs Service comparison . . . 11
- 9.2 Service definition . . . 11
- 9.3 Service server node . . . 11
- 9.4 Calling the service . . . 12

10 Gazebo World File 13

11 Launch File 13

12 Build and Launch 15
- 12.1 Validate the URDF . . . 15
- 12.2 Build . . . 15
- 12.3 Launch . . . 15

13 Drive the Robot 15
- 13.1 Keyboard teleoperation . . . 15
- 13.2 Custom service . . . 16
- 13.3 CLI verification . . . 16

14 Old Plugin vs ROS 2 Control 16

15 Troubleshooting 17

16 File Summary 17

17 Next Steps 18

§ 1 What Is ROS 2 Control?

ROS 2 Control is the official **hardware abstraction and controller management framework** for ROS 2. It solves a fundamental problem: every robot has different hardware, but high-level software (Nav2, MoveIt, your own nodes) should not care about those hardware details.

Without ROS 2 Control you write custom glue code for every robot. With it, you write a *hardware interface plugin* once and reuse standard, tested controllers across any robot — simulation or real.

> ROS 2 Control is not just for simulation. The same controller code that runs in Gazebo will run on your real robot. You only swap the hardware interface plugin.

1.1 Problems it solves

- **Hardware abstraction** — isolates controller logic from motor-driver code.
- **Real-time safety** — the update loop runs at a fixed rate (50–1000 Hz) using shared memory, not ROS topics.
- **Hot-swappable controllers** — start, stop, and switch controllers at runtime without restarting.
- **Portability** — the same YAML and controller nodes run on simulation and on the real robot.

§ 2 Architecture Overview

Layer	Component	Responsibility
High Level	Nav2 / MoveIt / Your node	Sends goals, publishes `/cmd_vel`
Middle	Controllers	Converts commands to joint signals
Middle	Controller Manager	Loads/starts/stops/switches controllers
Low Level	Hardware Interface	Reads/writes real motors or Gazebo

The data flow through the real-time loop:

```
Your Node / Nav2
  publishes /cmd_vel  (geometry_msgs/Twist)
        |
diff_drive_controller
  converts Twist  ->  left wheel speed, right wheel speed
        |
Controller Manager  (real-time loop @ 50 Hz)
        |
Hardware Interface
  GazeboSystem   (simulation)
  YourRobotHW    (real robot -- same controller, different plugin)
```

2.1 Key components

`controller_manager` The central node. Owns the real-time loop, loads controller plugins, manages their lifecycle (unconfigured → inactive → active).

Hardware Interface Plugin implementing `on_init`, `read`, `write`. In simulation: `GazeboSystem`. For real hardware: write your own (one-time effort).

Controller Plugin that reads *state interfaces* and writes *command interfaces* every update cycle.

`joint_state_broadcaster` A special controller that only reads state and publishes `/joint_states`. Required by `robot_state_publisher` and RViz.

2.2 Controller lifecycle

`unconfigured` → `inactive` → `active` → `inactive` → `finalized`

Switch controllers at runtime (no restart required):

```
ros2 control switch_controllers \
  --activate   diff_drive_controller \
  --deactivate my_other_controller
```

§ 3 Key Concepts

3.1 Command Interfaces vs State Interfaces

Interface	Direction	Meaning
`command/velocity`	Controller→HW	Set target wheel speed
`command/position`	Controller→HW	Set target joint angle
`command/effort`	Controller→HW	Set torque
`state/velocity`	HW→Controller	Report current speed
`state/position`	HW→Controller	Report current angle
`state/effort`	HW→Controller	Report torque

Two controllers cannot claim the same *command* interface simultaneously. The Controller Manager enforces this.

3.2 Controller types used in this tutorial

- `diff_drive_controller` — subscribes to `/cmd_vel` and converts it to left/right wheel velocity commands.
- `joint_state_broadcaster` — reads all state interfaces and publishes `sensor_msgs/JointState` to `/joint_states`.
- `joint_trajectory_controller` — for arms; not used here but easy to add later.

§ 4 Install Required Packages

```
# Core ROS 2 Control
sudo apt update
sudo apt install -y ros-humble-ros2-control
sudo apt install -y ros-humble-ros2-controllers

# Gazebo Classic 11 and ROS bridge
sudo apt install -y gazebo
sudo apt install -y ros-humble-gazebo-ros-pkgs

# Gazebo + ROS 2 Control integration plugin
sudo apt install -y ros-humble-gazebo-ros2-control

# State publishers
sudo apt install -y ros-humble-robot-state-publisher
sudo apt install -y ros-humble-joint-state-publisher
sudo apt install -y ros-humble-joint-state-publisher-gui
sudo apt install -y ros-humble-xacro

# Teleoperation
sudo apt install -y ros-humble-teleop-twist-keyboard

# Verify
source /opt/ros/humble/setup.bash
ros2 pkg prefix ros2_controllers        # must print a path
ros2 pkg prefix gazebo_ros2_control     # must print a path
```

§ 5 Create the ROS 2 Package

```
cd ~/ros2_ws/src

ros2 pkg create four_wheel_robot \
  --build-type ament_cmake \
  --dependencies rclcpp

cd four_wheel_robot
mkdir urdf launch worlds rviz config srv
```

Resulting layout:

```
four_wheel_robot/
  CMakeLists.txt        # build rules
  package.xml           # metadata + dependencies
  urdf/                 # robot description
  config/               # controllers.yaml
  srv/                  # custom service definition
  launch/               # launch file + service node
  worlds/               # Gazebo SDF world
  rviz/                 # optional RViz config
```

5.1 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(four_wheel_robot)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

# Generate custom service bindings
rosidl_generate_interfaces(${PROJECT_NAME}
```

```
  "srv/SetVelocity.srv"
)

# Install resource folders
install(DIRECTORY
  urdf launch worlds rviz config
  DESTINATION share/${PROJECT_NAME}
)

# Install Python service node as executable
install(PROGRAMS
  launch/velocity_service.py
  DESTINATION lib/${PROJECT_NAME}
)

ament_package()
```

Every directory in `install(DIRECTORY ...)` must *physically exist* before building. The `mkdir` above creates them all.

5.2 package.xml

```
<?xml version="1.0"?>
<package format="3">
  <n>four_wheel_robot</n>
  <version>0.0.1</version>
  <description>Four-wheel robot with ROS 2 Control</description>
  <maintainer email="you@email.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>rosidl_default_generators</buildtool_depend>

  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>joint_state_publisher</exec_depend>
  <exec_depend>gazebo_ros</exec_depend>
  <exec_depend>gazebo_ros2_control</exec_depend>
  <exec_depend>ros2_controllers</exec_depend>
  <exec_depend>xacro</exec_depend>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>

  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

§ 6 Write the URDF

Create `urdf/four_wheel_robot.urdf`.

6.1 Materials

```
<?xml version="1.0"?>
<robot name="four_wheel_robot">

```

```
  <material name="blue">
    <color rgba="0.2 0.4 0.8 1.0"/>
  </material>
  <material name="dark_gray">
    <color rgba="0.15 0.15 0.15 1.0"/>
  </material>
  <material name="black">
    <color rgba="0.05 0.05 0.05 1.0"/>
  </material>
```

6.2 Chassis (base_link)

```
  <!-- Chassis: 0.5 m long x 0.4 m wide x 0.1 m tall -->
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><box size="0.5 0.4 0.1"/></geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><box size="0.5 0.4 0.1"/></geometry>
    </collision>
    <inertial>
      <mass value="4.0"/>
      <!--
        Box inertia  (m=4, x=0.5, y=0.4, z=0.1):
          Ixx = m/12*(y^2+z^2) = 4/12*(0.16+0.01) = 0.0567
          Iyy = m/12*(x^2+z^2) = 4/12*(0.25+0.01) = 0.0867
          Izz = m/12*(x^2+y^2) = 4/12*(0.25+0.16) = 0.1367
      -->
      <inertia ixx="0.0567" ixy="0.0" ixz="0.0"
               iyy="0.0867" iyz="0.0"
               izz="0.1367"/>
    </inertial>
  </link>
```

Inertia formulas for a uniform solid box:

$$I_{xx} = \frac{m}{12}(y^2 + z^2), \quad I_{yy} = \frac{m}{12}(x^2 + z^2), \quad I_{zz} = \frac{m}{12}(x^2 + y^2)$$

6.3 Front wheels (fixed — passive)

The `diff_drive_controller` drives exactly one joint per side. Front wheels use `type="fixed"` — they move with the chassis with no actuation required.

```
  <!-- FRONT LEFT WHEEL -->
  <link name="front_left_wheel">
    <visual>
      <!-- rpy pi/2 rotates the cylinder onto its side so it rolls -->
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <!--
        Solid cylinder (m=0.5, r=0.1, h=0.05):
```

```
          Ixx=Iyy  = m/12*(3r^2+h^2)  =  0.5/12*(0.03+0.0025)  =  0.00135
          Izz      = m/2*r^2          =  0.5/2*0.01             =  0.00250
      -->
      <inertia ixx="0.00135" ixy="0.0" ixz="0.0"
               iyy="0.00135" iyz="0.0"
               izz="0.00250"/>
    </inertial>
  </link>
  <joint name="front_left_wheel_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="front_left_wheel"/>
    <origin xyz="0.15 0.225 -0.05" rpy="0 0 0"/>
  </joint>

  <!-- FRONT RIGHT WHEEL -->
  <link name="front_right_wheel">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.00135" ixy="0.0" ixz="0.0"
               iyy="0.00135" iyz="0.0"
               izz="0.00250"/>
    </inertial>
  </link>
  <joint name="front_right_wheel_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="front_right_wheel"/>
    <origin xyz="0.15 -0.225 -0.05" rpy="0 0 0"/>
  </joint>
```

6.4 Rear wheels (continuous — driven)

```
  <!-- REAR LEFT WHEEL -->
  <link name="rear_left_wheel">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.00135" ixy="0.0" ixz="0.0"
               iyy="0.00135" iyz="0.0"
               izz="0.00250"/>
    </inertial>
  </link>
  <joint name="rear_left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child  link="rear_left_wheel"/>
    <origin xyz="-0.15 0.225 -0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
```

```
  </joint>

  <!-- REAR RIGHT WHEEL -->
  <link name="rear_right_wheel">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1" length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.00135" ixy="0.0" ixz="0.0"
               iyy="0.00135" iyz="0.0"
               izz="0.00250"/>
    </inertial>
  </link>
  <joint name="rear_right_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child  link="rear_right_wheel"/>
    <origin xyz="-0.15 -0.225 -0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>
```

Joint origin summary:

Wheel	X (m)	Y (m)	Z (m)	Note
Front left	+0.15	+0.225	−0.05	forward, left
Front right	+0.15	−0.225	−0.05	forward, right
Rear left	−0.15	+0.225	−0.05	rear, left
Rear right	−0.15	−0.225	−0.05	rear, right

6.5 Lidar sensor link

```
  <!-- LIDAR (visual placeholder -- add ray sensor plugin later) -->
  <link name="lidar_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
      <material name="black"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.0001" ixy="0.0" ixz="0.0"
               iyy="0.0001" iyz="0.0"
               izz="0.0001"/>
    </inertial>
  </link>
  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="lidar_link"/>
```

```
    <origin xyz="0.1 0 0.07" rpy="0 0 0"/>
  </joint>
```

§ 7 Add the ros2_control Block to the URDF

Add this inside `<robot>` after all joints, before `</robot>`.

```
  <!-- ROS 2 CONTROL HARDWARE INTERFACE ===================== -->
  <ros2_control name="gazebo_system" type="system">
    <hardware>
      <!-- GazeboSystem: hardware plugin for Gazebo Classic -->
      <plugin>gazebo_ros2_control/GazeboSystem</plugin>
    </hardware>

    <!-- Only driven joints go here. Fixed joints are excluded. -->
    <joint name="rear_left_wheel_joint">
      <command_interface name="velocity"/>
      <state_interface   name="velocity"/>
      <state_interface   name="position"/>
    </joint>

    <joint name="rear_right_wheel_joint">
      <command_interface name="velocity"/>
      <state_interface   name="velocity"/>
      <state_interface   name="position"/>
    </joint>
  </ros2_control>

  <!-- GAZEBO PLUGIN: loads ros2_control into Gazebo ======== -->
  <gazebo>
    <plugin name="gazebo_ros2_control"
            filename="libgazebo_ros2_control.so">
      <parameters>
        $(find four_wheel_robot)/config/controllers.yaml
      </parameters>
    </plugin>
  </gazebo>

  <!-- GAZEBO SURFACE PROPERTIES ============================ -->
  <gazebo reference="base_link">
    <material>Gazebo/Blue</material>
  </gazebo>

  <gazebo reference="rear_left_wheel">
    <material>Gazebo/DarkGrey</material>
    <surface>
      <friction><ode><mu>1.0</mu><mu2>1.0</mu2></ode></friction>
      <contact><ode><kp>1e6</kp><kd>100</kd></ode></contact>
    </surface>
  </gazebo>

  <!-- Repeat identical friction block for rear_right_wheel, -->
  <!-- front_left_wheel, front_right_wheel                    -->

  <gazebo reference="lidar_link">
    <material>Gazebo/Black</material>
  </gazebo>

</robot>
```

Front wheels have `type="fixed"` joints and must **not** appear in the `<ros2_control>` block. The Gazebo plugin will error if you try to claim a fixed joint.

To add an arm later: add the arm joints to the `<ros2_control>` block with `position` command interfaces, then add a `joint_trajectory_controller` to `controllers.yaml`. No other changes needed.

§ 8 Controllers Configuration

Create `config/controllers.yaml`:

```yaml
controller_manager:
  ros__parameters:
    update_rate: 50    # control loop Hz

    diff_drive_controller:
      type: diff_drive_controller/DiffDriveController

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

diff_drive_controller:
  ros__parameters:
    # Must match joint names in the ros2_control URDF block
    left_wheel_names:  ["rear_left_wheel_joint"]
    right_wheel_names: ["rear_right_wheel_joint"]

    # Must match actual robot geometry
    wheel_separation: 0.45   # |y_left|+|y_right| = 0.225+0.225
    wheel_radius:     0.1    # cylinder radius from URDF

    odom_frame_id:  odom
    base_frame_id:  base_link
    publish_odom:    true
    publish_odom_tf: true

    cmd_vel_timeout: 0.5     # stop if no cmd_vel for 0.5 s

    linear.x.has_velocity_limits: true
    linear.x.max_velocity:   1.0    # m/s
    linear.x.min_velocity:  -1.0
    angular.z.has_velocity_limits: true
    angular.z.max_velocity:  2.0    # rad/s
    angular.z.min_velocity: -2.0
```

8.1 Differential drive kinematics

The controller converts a single `Twist` into wheel speeds:

$$v_{\text{left}} = v_x - \frac{\omega_z \cdot d}{2} \qquad v_{\text{right}} = v_x + \frac{\omega_z \cdot d}{2}$$

where v_x=`linear.x`, ω_z=`angular.z`, d=`wheel_separation`.

Command	Left (m/s)	Right (m/s)	Motion
$v_x = 1,\ \omega_z = 0$	+1.00	+1.00	Straight ahead
$v_x = 0,\ \omega_z = 1$	−0.23	+0.23	Spin left
$v_x = 0.5,\ \omega_z = 0.5$	+0.39	+0.61	Curve left

§ 9 Create a Custom ROS 2 Service

A service uses request/response communication — one node asks, another answers. This is cleaner than raw topic publishing when you want confirmation the command was received, and when you need timed commands that auto-stop the robot.

9.1 Topic vs Service comparison

	Topic (cmd_vel)	Service (set_velocity)
Pattern	Fire-and-forget	Request + response
Confirmation	None	Yes (`success` field)
Timed cmds	Needs external timer	Built in (`duration` field)
Best for	Continuous teleop	Discrete "go N seconds"

9.2 Service definition

Create `srv/SetVelocity.srv`:

```
# Request
float64 linear_x      # forward / backward  (m/s)
float64 angular_z     # rotation speed       (rad/s)
float64 duration      # seconds to apply; 0 = indefinite
---
# Response
bool     success
string   message
```

Run `colcon build` after adding a `.srv` file. The build system generates C++ and Python bindings automatically.

9.3 Service server node

Create `launch/velocity_service.py`:

```
#!/usr/bin/env python3
"""
velocity_service.py
Advertises /set_velocity.
Publishes Twist to /diff_drive_controller/cmd_vel.
Auto-stops after the requested duration if > 0.
"""
import threading
import rclpy
```

```python
from rclpy.node import Node
from geometry_msgs.msg import Twist
from four_wheel_robot.srv import SetVelocity


class VelocityService(Node):

    def __init__(self):
        super().__init__('velocity_service')

        self.pub = self.create_publisher(
            Twist, '/diff_drive_controller/cmd_vel', 10)

        self.srv = self.create_service(
            SetVelocity, 'set_velocity', self.handle_request)

        self.get_logger().info(
            'VelocityService ready on /set_velocity')

    def handle_request(self, request, response):
        self.get_logger().info(
            f'linear={request.linear_x:.2f}  '
            f'angular={request.angular_z:.2f}  '
            f'duration={request.duration:.2f}s')

        # Send the velocity command
        twist = Twist()
        twist.linear.x  = request.linear_x
        twist.angular.z = request.angular_z
        self.pub.publish(twist)

        # Auto-stop in background thread
        if request.duration > 0.0:
            def stop_later():
                import time
                time.sleep(request.duration)
                self.pub.publish(Twist())   # zero = stop
                self.get_logger().info('Auto-stopped.')
            threading.Thread(target=stop_later,
                             daemon=True).start()

        response.success = True
        response.message = (
            f'OK: linear={request.linear_x}, '
            f'angular={request.angular_z}')
        return response


def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(VelocityService())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

9.4 Calling the service

```bash
# Forward 0.5 m/s for 3 s, then auto-stop
ros2 service call /set_velocity four_wheel_robot/srv/SetVelocity \
  "{linear_x: 0.5, angular_z: 0.0, duration: 3.0}"

# Spin left in place for 2 s
```

```
ros2 service call /set_velocity four_wheel_robot/srv/SetVelocity \
  "{linear_x: 0.0, angular_z: 1.0, duration: 2.0}"

# Curve right indefinitely (duration 0 = no auto-stop)
ros2 service call /set_velocity four_wheel_robot/srv/SetVelocity \
  "{linear_x: 0.4, angular_z: -0.6, duration: 0.0}"

# Stop immediately
ros2 service call /set_velocity four_wheel_robot/srv/SetVelocity \
  "{linear_x: 0.0, angular_z: 0.0, duration: 0.0}"
```

Calling the service from another Python node:

```python
from four_wheel_robot.srv import SetVelocity

class MissionNode(Node):
    def __init__(self):
        super().__init__('mission_node')
        self.cli = self.create_client(SetVelocity, 'set_velocity')
        self.cli.wait_for_service(timeout_sec=5.0)

    def drive(self, vx, wz, duration=0.0):
        req = SetVelocity.Request()
        req.linear_x  = vx
        req.angular_z = wz
        req.duration  = duration
        future = self.cli.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        return future.result()    # SetVelocity.Response
```

§ 10 Gazebo World File

Create `worlds/empty.world`:

```xml
<?xml version="1.0"?>
<sdf version="1.6">
  <world name="default">
    <include><uri>model://sun</uri></include>
    <include><uri>model://ground_plane</uri></include>
    <physics type="ode">
      <max_step_size>0.001</max_step_size>
      <real_time_factor>1</real_time_factor>
      <real_time_update_rate>1000</real_time_update_rate>
    </physics>
  </world>
</sdf>
```

§ 11 Launch File

Create `launch/gazebo.launch.py`:

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess, TimerAction
from launch_ros.actions import Node

def generate_launch_description():
    pkg   = get_package_share_directory('four_wheel_robot')
    urdf  = os.path.join(pkg, 'urdf',   'four_wheel_robot.urdf')
```

```python
    world = os.path.join(pkg, 'worlds', 'empty.world')

    with open(urdf, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([

        # 1. Start Gazebo Classic
        ExecuteProcess(
            cmd=['gazebo', '--verbose', world,
                 '-s', 'libgazebo_ros_init.so',
                 '-s', 'libgazebo_ros_factory.so'],
            output='screen',
        ),

        # 2. Publish URDF on /robot_description
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{
                'robot_description': robot_description,
                'use_sim_time': True,
            }],
            output='screen',
        ),

        # 3. Spawn robot  (wait 3 s for Gazebo to load)
        TimerAction(period=3.0, actions=[
            Node(
                package='gazebo_ros',
                executable='spawn_entity.py',
                arguments=[
                    '-topic', '/robot_description',
                    '-entity', 'four_wheel_robot',
                    '-x', '0.0', '-y', '0.0', '-z', '0.15',
                ],
                output='screen',
            ),
        ]),

        # 4. Load controllers  (wait for robot to spawn)
        TimerAction(period=5.0, actions=[
            Node(package='controller_manager',
                 executable='spawner',
                 arguments=['joint_state_broadcaster'],
                 output='screen'),
            Node(package='controller_manager',
                 executable='spawner',
                 arguments=['diff_drive_controller'],
                 output='screen'),
        ]),

        # 5. Start velocity service node
        TimerAction(period=6.5, actions=[
            Node(
                package='four_wheel_robot',
                executable='velocity_service.py',
                name='velocity_service',
                output='screen',
            ),
        ]),
    ])
```

Startup sequence:

t (s)	Event	What happens
0.0	Gazebo starts	Physics engine and world load
0.0	RSP starts	`/robot_description` published immediately
3.0	Robot spawns	URDF model inserted into Gazebo
5.0	Controllers	`joint_state_broadcaster` + `diff_drive_controller` activate
6.5	Service starts	`/set_velocity` becomes available

§ 12 Build and Launch

12.1 Validate the URDF

```
source /opt/ros/humble/setup.bash
check_urdf ~/ros2_ws/src/four_wheel_robot/urdf/four_wheel_robot.urdf

# Expected:
#   robot name is: four_wheel_robot
#   ---------- Successfully Parsed XML ---------------
#   root Link: base_link has 5 children
```

12.2 Build

```
cd ~/ros2_ws
rm -rf build/four_wheel_robot install/four_wheel_robot
colcon build --packages-select four_wheel_robot
source install/setup.bash
```

12.3 Launch

```
ros2 launch four_wheel_robot gazebo.launch.py

# Confirm these lines appear in the terminal:
#   [Msg] Loaded plugin: libgazebo_ros2_control.so
#   [INFO] Loaded controller: joint_state_broadcaster
#   [INFO] Loaded controller: diff_drive_controller
#   [INFO] VelocityService ready on /set_velocity
```

§ 13 Drive the Robot

13.1 Keyboard teleoperation

```
# New terminal -- source workspace first
source /opt/ros/humble/setup.bash && source ~/ros2_ws/install/setup.bash

ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args --remap cmd_vel:=/diff_drive_controller/cmd_vel
```

Key	Action
`i`	Drive forward
`,`	Drive backward
`j`	Turn left
`l`	Turn right
`k`	Stop
`q` / `z`	Increase / decrease max speed

The topic is `/diff_drive_controller/cmd_vel`, **not** `/cmd_vel`. The controller namespace prefix is mandatory with ROS 2 Control.

13.2 Custom service

```
ros2 service call /set_velocity four_wheel_robot/srv/SetVelocity \
  "{linear_x: 0.5, angular_z: 0.0, duration: 3.0}"
```

13.3 CLI verification

```
# List controllers and state
ros2 control list_controllers
# joint_state_broadcaster  active
# diff_drive_controller    active

# List hardware interfaces
ros2 control list_hardware_interfaces

# Watch odometry
ros2 topic echo /diff_drive_controller/odom

# All active topics
ros2 topic list

# TF tree PDF
ros2 run tf2_tools view_frames && evince frames.pdf
```

§ 14 Old Plugin vs ROS 2 Control

Feature	Old Gazebo plugin	ROS 2 Control
Config location	Inside URDF `<gazebo>`	Separate YAML file
Command topic	`/cmd_vel`	`/diff_drive_controller/cmd_vel`
Controller swap	Restart required	Runtime hot-swap
Real hardware	Simulation only	Same code, swap plugin
Adding arm	Separate plugin	Add to same YAML
Lifecycle mgmt	None	Full ROS 2 lifecycle

§ 15 Troubleshooting

Symptom	Fix
`gazebo_ros2_control` not found	`sudo apt install ros-humble-gazebo-ros2-control`
Controller stuck at `inactive`	Increase `TimerAction` from 5.0 to 7.0 s
Robot falls through ground	Change spawn `-z` from `0.15` to `0.25`
No movement from `cmd_vel`	Use `/diff_drive_controller/cmd_vel` not `/cmd_vel`
Build fails on `srv` generation	Add `rosidl_default_generators` to both `CMakeLists.txt` and `package.xml`
`/set_velocity` not found	Check `velocity_service.py` is in `lib/four_wheel_robot/` and node started
`check_urdf` fails on `$(find)`	Expected – `check_urdf` can't resolve xacro macros; works fine at launch time
Wheels spin, robot stationary	Verify `wheel_separation` and `wheel_radius` match the URDF exactly
`spawn_entity` times out	Increase `TimerAction` from 3.0 to 5.0 s
Broken TF tree in RViz	Confirm `joint_state_broadcaster` is active

§ 16 File Summary

File	Purpose
`package.xml`	Package metadata and runtime dependencies
`CMakeLists.txt`	Build rules; generates `SetVelocity`
`srv/SetVelocity.srv`	Custom service definition
`urdf/four_wheel_robot.urdf`	Geometry, inertia, joints, ros2_control block
`config/controllers.yaml`	Controller types and parameters
`worlds/empty.world`	Gazebo SDF with ground and physics
`launch/gazebo.launch.py`	Orchestrates all five startup steps
`launch/velocity_service.py`	Custom service server node

§ 17 Next Steps

Topic	What to do
Xacro	Refactor the four identical wheel blocks into a `xacro:macro` — shrinks URDF from 200 to 80 lines
Arm control	Add arm joints to `<ros2_control>` with `position` interfaces; add `joint_trajectory_controller` to YAML
Lidar plugin	Add `libgazebo_ros_ray_sensor.so` to publish `sensor_msgs/LaserScan` on `/scan`
Navigation (Nav2)	Feed `/odom` and `/scan` into Nav2 for autonomous waypoint following
Real hardware	Replace `GazeboSystem` with your motor driver plugin — controllers remain identical
RViz	Add an RViz node to the launch file for live model, TF, and laser visualisation